# Heshbonit Project Documentation

Generated from the real code in `Heshbonit` and `LLM-test` on 2026-04-24.

This document describes only behavior that is present in the checked-out code. Where a file, table, or function exists but is not wired into an observed runtime path, that is called out explicitly.

## Repository Map

The project consists of two repositories that work together:

| Repository | Role | Main stack | Entry point |
|---|---|---|---|
| `Heshbonit` | Browser application for uploading, reviewing, editing, and saving invoice records | React 19, Vite | `src/main.jsx`, `src/App.jsx` |
| `LLM-test` | HTTP API for invoice extraction, Gmail synchronization, file retrieval, and SQLite persistence | Express 4, TypeScript, better-sqlite3, OpenAI SDK, googleapis | `src/app.ts` |

The frontend points to the backend through environment variables and Vite proxy configuration:

- `Heshbonit/.env.production` sets `VITE_API_URL=https://llm-test-production.up.railway.app/api/invoices/upload` and `VITE_GMAIL_API_URL=https://llm-test-production.up.railway.app/api/gmail`.
- `Heshbonit/vite.config.js` proxies local `/api` requests to `https://llm-test-production.up.railway.app`.
- `Heshbonit/src/App.jsx` derives `API_URL`, `API_BASE`, and `GMAIL_API_BASE` from those values or from local `/api/...` defaults.

## Architecture Overview

At runtime, the application is a single-page React UI backed by an Express API. The backend handles four major responsibilities:

1. Accept invoice files from the browser.
2. Convert PDFs to images when needed.
3. Send image data to OpenAI for invoice field extraction.
4. Store and retrieve invoice records and original file data in SQLite.

Gmail synchronization is a second ingestion path. The backend uses Google OAuth, reads messages from a configured Gmail label, downloads attachments, or creates a simple PDF from

message text when no attachment exists. Each resulting file then passes through the same invoice processing service used for uploaded files.

Text architecture diagram:

```
Browser user
  |
  v
Heshbonit React app
  |-- GET /api/invoices/list ----------------------.
  |-- POST /api/invoices/upload multipart --------.|
  |-- POST /api/invoices/save-batch JSON --------.||
  |-- GET /api/invoices/file/:id ----------------.|||
  |-- GET /api/gmail/connect / callback --------.||||
  |-- POST /api/gmail/sync --------------------.|||||
                                                vvvvv
                                          LLM-test Express API
                                            |-- invoice routes
                                            |-- gmail routes
                                            |-- multer upload middleware
                                            |-- PDF conversion service
                                            |-- OpenAI extraction service
                                            |-- SQLite invoice service
```

The backend process starts in `LLM-test/src/app.ts`: it loads `.env`, initializes the database, configures CORS and large JSON/urlencoded bodies, serves `public`, exposes `/health`, and mounts `/api/invoices` and `/api/gmail`.

# Frontend Application

## Entry Point

`Heshbonit/src/main.jsx` mounts `<App />` into `document.getElementById('root')` under React `StrictMode`.

The application logic is concentrated in `Heshbonit/src/App.jsx`. There are no separate React component modules in the current codebase; the UI, state management, transformation helpers, upload flow, Gmail flow, table editing, duplicate checks, and save operation all live inside `App`.

## Runtime Configuration

`App.jsx` defines:

- `API_URL`: upload endpoint, defaulting to `/api/invoices/upload`.
- `API_BASE`: invoice API base, derived by removing `/upload` from `VITE_API_URL`, defaulting to `/api/invoices`.
- `GMAIL_API_BASE`: Gmail API base, defaulting to `/api/gmail`.

These constants are used by the upload, list, file, save, and Gmail sync flows.

## State Model

The `App` component keeps the following state:

| State | Purpose |
|---|---|
| `processing` | Shows upload/extraction progress and disables upload controls. |
| `result` | The table data. It contains stored rows from the database, newly uploaded rows, Gmail rows, and failed rows. |
| `error` | Displays backend, parsing, duplicate, or sync errors. |
| `selectedRows` | Tracks selected table row indexes for bulk actions. |
| `saving` | Shows save progress and disables the save action. |
| `editingCell` | Tracks the active editable table cell. |
| `gmailSummary` | Stores a count after Gmail sync. |
| `gmailLoading` | Shows Gmail sync progress and disables Gmail sync. |
| `dbLoaded` | Prevents Gmail callback auto-sync until database rows have loaded. |
| `uploadInputRef`, `cameraInputRef` | Hidden file input triggers for regular upload and camera capture. |

## Data Transformation Helpers

`parseDisplayDate` normalizes `/` and `.` separators to `-`, expects three date parts in day-month-year order, and returns a JavaScript `Date` or `null`.

`sortResultsByDateAsc` clones and sorts rows by parsed date. Failed rows are pushed after successful rows. Rows without parseable dates are also pushed later than dated rows.

`normalizeAmount` returns an empty string for nullish/empty input, parses numeric strings, and formats valid numbers with two decimals.

`displayDateToISO` turns display dates into `YYYY-MM-DD` for duplicate comparison.

`buildDuplicateKey` combines ISO date and normalized total as `date|total`; it returns `null` when either value is unavailable.

`base64ToBlobUrl` converts base64 file content returned by Gmail sync into a browser object URL.

`mapInvoiceFromDatabase` converts backend invoice records into table rows. It maps backend fields such as `vendorName`, `totalWithoutVat`, `vat`, and `totalWithVat` into UI fields named `supplier`, `payment`, `vat`, and `total`; it also creates a file URL using `/api/invoices/file/:id`.

## Initial Database Load

On mount, `App.jsx` calls `GET ${API_BASE}/list`. If the response is successful and includes `invoices`, the app maps each invoice through `mapInvoiceFromDatabase`, sorts the rows, and stores them in `result`. The `dbLoaded` flag is set in a `finally` block.

## Gmail Callback Handling

A second effect waits for `dbLoaded`. If the browser URL includes `gmail_connected=1`, the app removes that query parameter using `window.history.replaceState` and calls `handleGmailSync`.

This means the OAuth callback return URL is expected to route the browser back to the frontend, with `gmail_connected=1` appended.

## File Upload Flow

The UI has two hidden file inputs:

- One accepts `image/*,.pdf` and supports multiple files.
- One accepts `image/*` with `capture="environment"` for camera capture.

`handleFileChange` converts the `FileList` to an array, passes it to `processFiles`, then clears the input value so the same file can be selected again later.

`processFiles`:

1. Returns immediately when no files are selected.
2. Sets `processing`, clears row selection, and clears `error`.
3. Builds `FormData` with field name `invoices`.
4. Creates local object URLs for immediate file viewing.
5. Sends `POST ${API_URL}` with the multipart body.
6. Throws an error if the HTTP response is not OK or `json.success` is false.
7. Converts each backend result into a table row.
8. Marks failed files as `failed: true`.
9. For successful files, maps `vendorName`, `date`, `totalWithVat`, `totalWithoutVat`, and `confidence`; computes `vat` as `totalWithVat - totalWithoutVat`; marks the row as dirty and source `upload`.
10. Appends the rows to existing `result` and sorts.

## Gmail Sync Flow

`handleGmailSync`:

1. Sets `gmailLoading` and clears `error`.
2. Calls `POST ${GMAIL_API_BASE}/sync`.
3. If the backend returns `401` or `Gmail not connected`, redirects the browser to `${GMAIL_API_BASE}/connect`.
4. Throws an error if the response is not successful.

5. Stores a summary count from `json.total` or result length.
6. Converts each Gmail result into the same table shape used by upload results.
7. Converts returned base64 file data into a Blob URL when available.
8. Appends and sorts rows.

## Table Editing and Bulk Actions

The table supports row selection by index, select-all, inline editing, and bulk transforms:

- `toggleRow` adds/removes a row index from `selectedRows`.
- `toggleAll` selects all current rows or clears selection.
- `handleCopyWithoutVat` copies `total` into `payment`, sets `vat` to `0`, and marks selected successful rows dirty.
- `handleMarkPrinted` sets selected successful rows' `printed` field to yes.
- `handleApplyFraction` multiplies selected successful rows' `payment`, `vat`, and `total` by one of `2/3`, `1/2`, `1/3`, or `1/4`.
- `handleDeleteSelected` removes selected rows from the client table and revokes blob URLs for removed rows when no remaining row uses the same URL.
- `updateRowValue` updates `supplier`, numeric fields, or `date`, and marks the row dirty.
- `renderEditableCell` switches a cell between display text and an input when clicked. Enter and Escape both close the editor.

## Saving to Database

`handleSaveToDatabase` synchronizes the client table to the backend:

1. Sets `saving` and clears `error`.
2. Builds duplicate groups using `buildDuplicateKey`.
3. Rejects saving when a new non-stored row has the same date and total as an existing stored row.
4. Converts display dates into ISO dates.
5. Filters out failed rows.
6. Builds the JSON payload for `POST ${API_BASE}/save-batch`.
7. Sends fields including `id`, `fileName`, `mimeType`, `fileData` for new rows, `vendorName`, `date`, `totalWithVat`, `totalWithoutVat`, `vat`, `printed`, `currency`, and `confidence`.
8. Reloads the database list after a successful save.
9. Displays an alert containing `savedCount` and `deletedCount`.

The frontend deliberately omits `fileData` for rows already stored in the database. The backend update path only updates `fileData` when it is present.

# Backend Application

## Express Bootstrap

`LLM-test/src/app.ts` performs these startup actions:

- Calls `dotenv.config()`.
- Calls `initializeDatabase()`.
- Creates an Express app.
- Sets `PORT` from `process.env.PORT` or `8080`.
- Enables CORS.
- Allows JSON and URL-encoded request bodies up to `50mb`.
- Serves static files from `../public`.
- Defines `GET /health`.
- Mounts `invoiceRouter` at `/api/invoices`.
- Mounts `gmailRouter` at `/api/gmail`.
- Calls `app.listen(PORT)`.

## Upload Middleware

`LLM-test/src/middleware/upload.ts` configures Multer:

- Storage: in-memory buffers.
- Accepted MIME types: `image/jpeg`, `image/png`, `image/webp`, `application/pdf`.
- Rejected types receive `Invalid file type. Only JPG, PNG, WEBP and PDF are allowed.`
- File size limit: `10 * 1024 * 1024`.

The invoice upload route applies this middleware as `upload.array('invoices', 20)`, so the public multipart field name is `invoices`, and the route accepts up to 20 files.

## Invoice Routes

All routes below are mounted under `/api/invoices`.

### `POST /upload`

Accepts multipart files in field `invoices`.

Processing:

1. Rejects empty uploads with HTTP 400.
2. Processes all files with `Promise.all`.
3. For each file, starts with the uploaded buffer and MIME type.
4. If MIME type is `application/pdf`, converts the first PDF page to PNG with `convertPdfToImage`.
5. Calls `extractInvoiceData` with the image buffer and image MIME type.

6. Returns the original uploaded file as base64 in `fileData`.
7. Captures per-file errors into failed result objects rather than failing the whole batch.

Success response shape:

```
{
  "success": true,
  "total": 2,
  "results": [
    {
      "success": true,
      "filename": "invoice.pdf",
      "mimeType": "application/pdf",
      "data": {
        "vendorName": "Vendor",
        "date": "2026-04-24",
        "totalWithVat": 118,
        "totalWithoutVat": 100,
        "currency": "ILS",
        "confidence": "high"
      },
      "fileData": "base64..."
    }
  ]
}
```

Failed file result shape:

```
{
  "success": false,
  "filename": "bad-file.pdf",
  "error": "Failure message"
}
```

**`POST /save-batch`**

Accepts JSON `{ "invoices": [...] }`.

Processing:

1. Rejects non-array `invoices` with HTTP 400.
2. Loads existing invoices.
3. Deletes every existing database invoice whose `id` is absent from the incoming list.
4. For each incoming invoice:

   - If it has an `id`, checks `hasInvoiceChanges`. - Updates only changed existing invoices. - If it has no `id`, inserts it with `saveInvoice`.
5. Returns `savedCount` and `deletedCount`.

Important behavior: this is a synchronization endpoint, not append-only storage. Rows deleted in the frontend table are deleted from SQLite when they are absent from the incoming list.

#### `GET /list`

Returns all invoices from SQLite as `{ success: true, invoices }`. The query orders by `date ASC, createdAt ASC` when `createdAt` exists, otherwise by `date ASC, id ASC`.

#### `GET /file/:id`

Reads `fileData` and `mimeType` by invoice ID. It rejects non-numeric IDs with HTTP 400, returns HTTP 404 when file data is missing, decodes base64 file data to a buffer, sets `Content-Type`, sets `Content-Disposition: inline`, and sends the file.

#### `GET /stats`

Builds simple statistics from all invoices:

- `total`: invoice count.
- `totalRevenue`: sum of `totalWithVat`.
- `totalVat`: sum of `vat`.
- `avgConfidence`: hard-coded to `medium`.

The current frontend code does not call this route.

#### `POST /reset-db`

Deletes all invoice rows using `clearAllInvoices`. The current frontend code does not call this route.

#### `GET /reset-db-file`

Closes and deletes the SQLite database file, returning a message that the service should be restarted to recreate a clean database. The current frontend code does not call this route.

## Gmail Routes

All routes below are mounted under `/api/gmail`.

#### `GET /connect`

Redirects to the URL returned by `getAuthUrl`, which is generated by the Google OAuth client using scope `https://www.googleapis.com/auth/gmail.readonly`, `access_type: offline`, and `prompt: consent`.

#### `GET /callback`

Reads `code` from the query string. If missing, returns HTTP 400. Otherwise, it calls `handleOAuthCallback(code)`, stores tokens in process memory, and redirects to `GMAIL_RETURN_URL`

with `gmail_connected=1` appended when `GMAIL_RETURN_URL` exists. If that environment variable is absent, it sends `Gmail connected successfully`.

Tokens are stored in the module-level `tokens` variable in `gmailService.ts`; there is no persistent token storage in the current code.

#### `POST /sync`

Calls `fetchEmails`, then processes each message:

- If the message has attachments, each attachment is downloaded and processed through `processInvoiceFile`.
- If the message has no attachments, the backend creates a simple PDF from subject, sender, and snippet, then processes that generated PDF through `processInvoiceFile`.
- Each successful result is marked with `source: 'gmail'`.
- Per-item processing failures are returned as failed result objects.
- If Gmail has not been connected, returns HTTP 401 with `{ success: false, error: 'Gmail not connected' }`.

# Service Modules

## OpenAI Extraction Service

`LLM-test/src/services/openai.ts` creates an OpenAI client using `process.env.OPENAI_API_KEY`.

`extractInvoiceData(fileBuffer, mimeType)`:

1. Requires the MIME type to start with `image/`.
2. Converts the image buffer to a base64 data URL.
3. Calls `client.responses.create`.
4. Uses model `gpt-5.4`.
5. Sends one `input_image` with `detail: 'high'` and one `input_text` containing `EXTRACTION_PROMPT`.
6. Sets `max_output_tokens: 500` and `temperature: 0`.
7. Reads `response.output_text`.
8. Removes markdown JSON fences if present.
9. Parses JSON directly or extracts the first `{...}` block and parses that.

The prompt instructs the model to return only:

```
{
  "vendorName": "...",
  "date": "YYYY-MM-DD",
  "totalWithVat": 0,
  "totalWithoutVat": 0,
```

```
  "currency": "ILS",
  "confidence": "high"
}
```

The actual TypeScript interface allows nullable vendor, date, totals, and currency, with confidence restricted to `high`, `medium`, or `low`.

## PDF Conversion Service

`LLM-test/src/services/pdf.ts` uses `pdf2pic`:

- Source: PDF buffer.
- Density: `200`.
- Format: `png`.
- Width: `1654`.
- Height: `2339`.
- Page: `1`.
- Response type: `buffer`.

After conversion, it checks that the result buffer exists and validates PNG magic bytes `89 50 4E 47`. It throws errors for empty conversion output or invalid image headers. The Dockerfile and Nixpacks config both install GraphicsMagick and Ghostscript, which this service needs.

## Shared Invoice File Processor

`LLM-test/src/services/processInvoiceFile.ts` is used by the Gmail route.

It detects PDFs by MIME type, `.pdf` filename suffix, or `%PDF-` file signature. PDFs are converted to PNG and validated again with a PNG header check. It then calls `extractInvoiceData` and returns:

- `success: true`
- `filename`
- original `mimeType`
- extracted `data`
- original file as base64 `fileData`

On extraction errors, it wraps the message with filename, input MIME, processed image size, and original MIME.

The upload route has similar PDF/extraction logic inline instead of calling this shared processor.

## Gmail Service

`LLM-test/src/services/gmailService.ts` defines:

- `getAuthUrl`: generates Google OAuth URL.
- `handleOAuthCallback`: exchanges the OAuth code for tokens and stores them in memory.

- `getGmailClient`: throws `Gmail not connected` when there are no tokens.
- `fetchEmails`: finds the exact Gmail label named by `GMAIL_LABEL_NAME` or `Heshbonit`, lists up to 20 messages using `q: label:<labelName>`, loads full message payloads, collects attachments recursively, and filters messages to those whose `labelIds` include the exact label ID.
- `downloadAttachment`: downloads a Gmail attachment and decodes Gmail's URL-safe base64 format.
- `createSimplePdfBuffer`: creates a minimal PDF from text lines using a hand-built PDF string.

## Database Initialization

`LLM-test/src/database/db.ts` stores the SQLite database at `/app/data/invoices.db`. It creates `/app/data` if needed, opens the database with `better-sqlite3`, enables foreign keys, and logs the path.

`initializeDatabase` creates two tables:

### `invoices`

Columns created by the table definition:

- `id`
- `fileName`
- `mimeType`
- `fileData`
- `vendorName`
- `date`
- `totalWithVat`
- `totalWithoutVat`
- `vat`
- `currency`
- `confidence`
- `status`
- `createdAt`
- `updatedAt`

After table creation, `ensureColumn` also ensures `printed` exists.

### `gmail_staging`

Columns created by the table definition:

- `id`
- `gmailMessageId`
- `threadId`
- `fromAddress`

- `subject`
- `snippet`
- `receivedAt`
- `hasAttachments`
- `attachmentNames`
- `category`
- `isRelevant`
- `confidence`
- `reason`
- `createdAt`

The code also creates index `idx_gmail_message` on `gmailMessageId`, and ensures `gmailAttachmentId`, `fileName`, `mimeType`, and `sourceType` columns exist.

Observed usage note: no current route or service writes to or reads from `gmail_staging`. It is initialized but not otherwise used by the checked code.

`resetDatabaseFile` closes the database connection and deletes `/app/data/invoices.db` when it exists.

## Invoice Database Service

`LLM-test/src/database/invoiceService.ts` wraps SQLite access.

Major functions:

- `validateFileData`: rejects base64 file data larger than 10 MB.
- `getInvoiceColumns`: reads `PRAGMA table_info(invoices)` and returns a set of available column names.
- `normalizeComparableValue`: normalizes `undefined` and empty string to `null`.
- `saveInvoice`: validates file data, dynamically includes optional columns that exist, inserts a row, and returns the new ID.
- `getInvoiceById`: returns one invoice or `null`.
- `hasInvoiceChanges`: compares incoming invoice fields with the existing database row and returns whether an update is needed.
- `updateInvoice`: updates a row by ID, includes `fileData` only when provided, updates optional columns when they exist, and sets `updatedAt = CURRENT_TIMESTAMP` when that column exists.
- `deleteInvoice`: deletes by ID and returns whether a row changed.
- `saveBatch`: inserts all invoices and returns IDs. This function exists but is not called by the current routes.
- `syncInvoices`: updates rows with IDs and inserts rows without IDs. This function exists but is not called by the current routes.
- `getInvoices`: returns all invoices ordered by date and created time or ID.
- `clearAllInvoices`: deletes all invoice rows.

- `getInvoiceFileData`: returns `fileData` and `mimeType` for a single invoice ID.

# API and Interface Reference

## Frontend-to-Backend Interfaces

| Frontend action | Function | HTTP interface |
|---|---|---|
| Load existing rows | `loadDataFromDatabase` | `GET /api/invoices/list` |
| Upload files | `processFiles` | `POST /api/invoices/upload`, multipart field `invoices` |
| Save current table | `handleSaveToDatabase` | `POST /api/invoices/save-batch`, JSON |
| Open stored file | table file link | `GET /api/invoices/file/:id` |
| Start Gmail OAuth | `handleGmailSync` redirect path | `GET /api/gmail/connect` |
| Sync Gmail | `handleGmailSync` | `POST /api/gmail/sync` |

## Invoice Data Interface

Backend interface in `LLM-test/src/types/invoice.ts`:

```
interface InvoiceData {
  vendorName: string | null;
  date: string | null;
  totalWithVat: number | null;
  totalWithoutVat: number | null;
  currency: string | null;
  confidence: 'high' | 'medium' | 'low';
}
```

Upload success wraps this as `InvoiceResponse` with `success`, `filename`, `mimeType`, and `data`. The actual upload route also returns `fileData` as base64.

### Environment Variables Observed in Code

| Variable | Used by | Purpose |
|---|---|---|
| `VITE_API_URL` | frontend | Upload endpoint; also used to derive invoice API base. |
| `VITE_GMAIL_API_URL` | frontend | Gmail API base. |
| `OPENAI_API_KEY` | backend OpenAI service | Authenticates OpenAI API calls. |
| `PORT` | backend app | Express listen port, default `8080`. |
| `GOOGLE_CLIENT_ID` | Gmail service | Google OAuth client ID. |
| `GOOGLE_CLIENT_SECRET` | Gmail service | Google OAuth client secret. |
| `GOOGLE_REDIRECT_URI` | Gmail service | Google OAuth callback URL. |
| `GMAIL_RETURN_URL` | Gmail callback route | Frontend return URL after OAuth. |
| `GMAIL_LABEL_NAME` | Gmail service | Exact Gmail label to sync, default `Heshbonit`. |

Only `OPENAI_API_KEY` and `PORT` appear in `LLM-test/.env.example`; the Gmail variables are required by code paths but are not listed in that example file.

## User Flows by Component

### Flow 1: Opening the App and Loading Saved Invoices

1. Browser loads the Vite/React app.
2. `main.jsx` renders `App`.
3. `App` performs `GET /api/invoices/list`.
4. Backend `invoiceRouter.get('/list')` calls `getInvoices`.
5. `getInvoices` queries SQLite ordered by date.
6. Frontend `mapInvoiceFromDatabase` converts each record into UI row fields.
7. The table displays rows with totals and file links.

Components/modules involved:

- `Heshbonit/src/main.jsx`
- `Heshbonit/src/App.jsx`
- `LLM-test/src/routes/invoices.ts`
- `LLM-test/src/database/invoiceService.ts`
- `LLM-test/src/database/db.ts`

## Flow 2: Uploading Invoice Images or PDFs

1. User clicks upload or camera button.
2. Hidden file input opens.
3. `handleFileChange` calls `processFiles`.
4. Frontend sends multipart `POST /api/invoices/upload`.
5. Multer validates MIME type and size, storing files in memory.
6. Route converts PDFs to images with `convertPdfToImage`.
7. Route sends image data to `extractInvoiceData`.
8. OpenAI returns JSON fields.
9. Backend returns per-file result objects and base64 original files.
10. Frontend maps successful results into editable rows and failed results into error rows.

Components/modules involved:

- `App.jsx` upload controls and `processFiles`
- `upload.ts`
- `invoices.ts`
- `pdf.ts`
- `openai.ts`
- `types/invoice.ts`

## Flow 3: Reviewing and Correcting Extracted Rows

1. User sees extracted rows in the results table.
2. User clicks supplier, date, or amount fields.
3. `renderEditableCell` switches a cell into an input.
4. `updateRowValue` updates the field and marks the row dirty.
5. User may select rows and apply bulk actions.
6. Summary cards recompute totals directly from non-failed `result` rows.

Components/modules involved:

- `App.jsx` table rendering
- `renderEditableCell`
- `updateRowValue`
- `handleCopyWithoutVat`
- `handleMarkPrinted`
- `handleApplyFraction`
- `handleDeleteSelected`

## Flow 4: Saving the Table to SQLite

1. User clicks the save/update database button.
2. `handleSaveToDatabase` checks for duplicates between existing stored records and new rows using date plus total.
3. Failed rows are excluded.
4. Frontend posts `{ invoices }` to `/api/invoices/save-batch`.
5. Backend deletes existing rows omitted from the incoming list.
6. Backend updates changed rows with IDs.
7. Backend inserts rows without IDs.
8. Frontend reloads `/api/invoices/list`.
9. The reloaded database records replace the table state.

Components/modules involved:

- `App.jsx`
- `invoiceRouter.post('/save-batch')`
- `getInvoices`
- `deleteInvoice`
- `hasInvoiceChanges`
- `updateInvoice`
- `saveInvoice`

## Flow 5: Opening a Stored File

1. Stored rows have `fileUrl` set to `/api/invoices/file/:id`.
2. User clicks the file link.
3. Backend validates the ID.
4. Backend loads base64 file data and MIME type.
5. Backend sends decoded file bytes inline.

Components/modules involved:

- `mapInvoiceFromDatabase`
- table file link rendering
- `invoiceRouter.get('/file/:id')`
- `getInvoiceFileData`

## Flow 6: Gmail Connection and Sync

1. User clicks Gmail sync.
2. Frontend calls `POST /api/gmail/sync`.
3. If backend has no OAuth tokens, it returns HTTP 401.

4. Frontend redirects to `/api/gmail/connect`.
5. Backend redirects to Google OAuth.
6. Google redirects to `/api/gmail/callback`.
7. Backend exchanges the code for tokens.
8. Backend redirects to `GMAIL_RETURN_URL?gmail_connected=1` when configured.
9. Frontend sees `gmail_connected=1`, removes the query param, and calls sync again.
10. Backend fetches Gmail messages from the exact configured label.
11. Attachments are downloaded and processed; messages without attachments are converted to simple PDFs and processed.
12. Frontend appends Gmail-sourced rows to the table.

Components/modules involved:

- `App.jsx` `handleGmailSync`
- `gmail.ts`
- `gmailService.ts`
- `processInvoiceFile.ts`
- `pdf.ts`
- `openai.ts`

## Deployment and Runtime Notes From Code

The backend Dockerfile uses `node:20-slim`, installs `graphicsmagick` and `ghostscript`, runs `npm install`, builds TypeScript, exposes `8080`, and starts `node dist/app.js`.

`railway.toml` uses Nixpacks, starts with `npm run start`, and defines a volume mounted at `/app/data`. This matches the SQLite path hard-coded in `db.ts`.

`nixpacks.toml` installs `graphicsmagick` and `ghostscript`, matching the PDF conversion dependency.

The frontend production environment points to the Railway backend URL.

## Observed Gaps and Code Facts

These are not assumptions; they are direct observations from the current code:

- The frontend is a single large `App.jsx` module rather than a component tree split across files.
- The upload route has its own file-processing logic, while Gmail uses `processInvoiceFile`; these paths are similar but not unified.
- `gmail_staging` is created but not used by any observed route or service.
- Gmail OAuth tokens are stored only in memory, so a backend process restart loses the Gmail connection.

- `POST /save-batch` deletes database invoices that are missing from the incoming client payload.
- `GET /stats`, `POST /reset-db`, and `GET /reset-db-file` exist in the backend but are not called by the current frontend.
- The backend database path is `/app/data/invoices.db`, which is deployment-oriented. The code does not define a different local development database path.
- `LLM-test/.env.example` lists only `OPENAI_API_KEY` and `PORT`, while Gmail code also reads Google OAuth and Gmail label/return variables.

## Source Reference Index

Frontend:

- `Heshbonit/src/main.jsx`: React mount.
- `Heshbonit/src/App.jsx`: API constants, app state, data mapping, upload, Gmail sync, editing, bulk actions, save, and rendering.
- `Heshbonit/.env.production`: production backend URLs.
- `Heshbonit/vite.config.js`: local `/api` proxy.

Backend:

- `LLM-test/src/app.ts`: Express bootstrap and route mounting.
- `LLM-test/src/routes/invoices.ts`: invoice HTTP API.
- `LLM-test/src/routes/gmail.ts`: Gmail HTTP API.
- `LLM-test/src/middleware/upload.ts`: Multer upload constraints.
- `LLM-test/src/services/openai.ts`: OpenAI extraction prompt and API call.
- `LLM-test/src/services/pdf.ts`: PDF-to-PNG conversion.
- `LLM-test/src/services/processInvoiceFile.ts`: shared Gmail invoice file processing.
- `LLM-test/src/services/gmailService.ts`: OAuth, Gmail listing, attachment download, simple PDF creation.
- `LLM-test/src/database/db.ts`: SQLite initialization and schema creation.
- `LLM-test/src/database/invoiceService.ts`: invoice persistence operations.
- `LLM-test/src/types/invoice.ts`: invoice data interfaces.
- `LLM-test/Dockerfile`, `railway.toml`, `nixpacks.toml`: deployment/runtime dependencies.